Anexos Técnicos del Motor de Asignación Automática de Horarios de Clases

Documento 2 de la Entrega Final del Módulo 3. Diagramas UML Finales y Modelo de Datos

Jesús David Beltrán Zárate

Juan Sebastián Albarracín Isaza

Daniel Felipe Delgadillo Sepúlveda

Martín Daniel Porto Gutiérrez

Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Universidad El Bosque

Proyecto Núcleo 2 (15319), 2026-2, Grupo 5

Ing. Samuel Galindo Pinzón

5 de noviembre de 2026

Repositorio https://github.com/jesusBeltran79/ProgHorarioClase

Normas APA 7.ª edición

Introducción

Este documento reúne los diagramas finales del sistema construido. Reemplaza al Documento de Anexos del Módulo 2 y conserva su numeración para que las citas del Documento 1. Informe de Ingeniería (PN2_Grupo5_Doc1_Informe.pdf) y del Documento 3. Manual Técnico (PN2_Grupo5_Doc3_ManualTecnico.pdf) sigan siendo válidas. Cada diagrama se actualizó contra el código de la rama principal del repositorio. Los nombres de clases, métodos, tipos y tablas son los del código. Las fuentes en PlantUML están en la carpeta docs/diagramas.

Frente al Módulo 2 hay tres cambios en el conjunto de diagramas. El diagrama de clases se dividió en dos figuras porque el sistema construido tiene una capa de aplicación con servicios, seguridad y repositorios que no cabía en una sola página. El diagrama de secuencia ahora sigue una generación real a través de la API con el token JWT. Se agregó el diagrama de despliegue en contenedores, que en el Módulo 2 estaba combinado con el de componentes. La Tabla 1 lista los artefactos. Al final del módulo se agregó la Figura A4 con el módulo de usuarios por perfil y el diagrama entidad relación y el de componentes incluyen sus cambios.

Tabla 1

Índice de artefactos

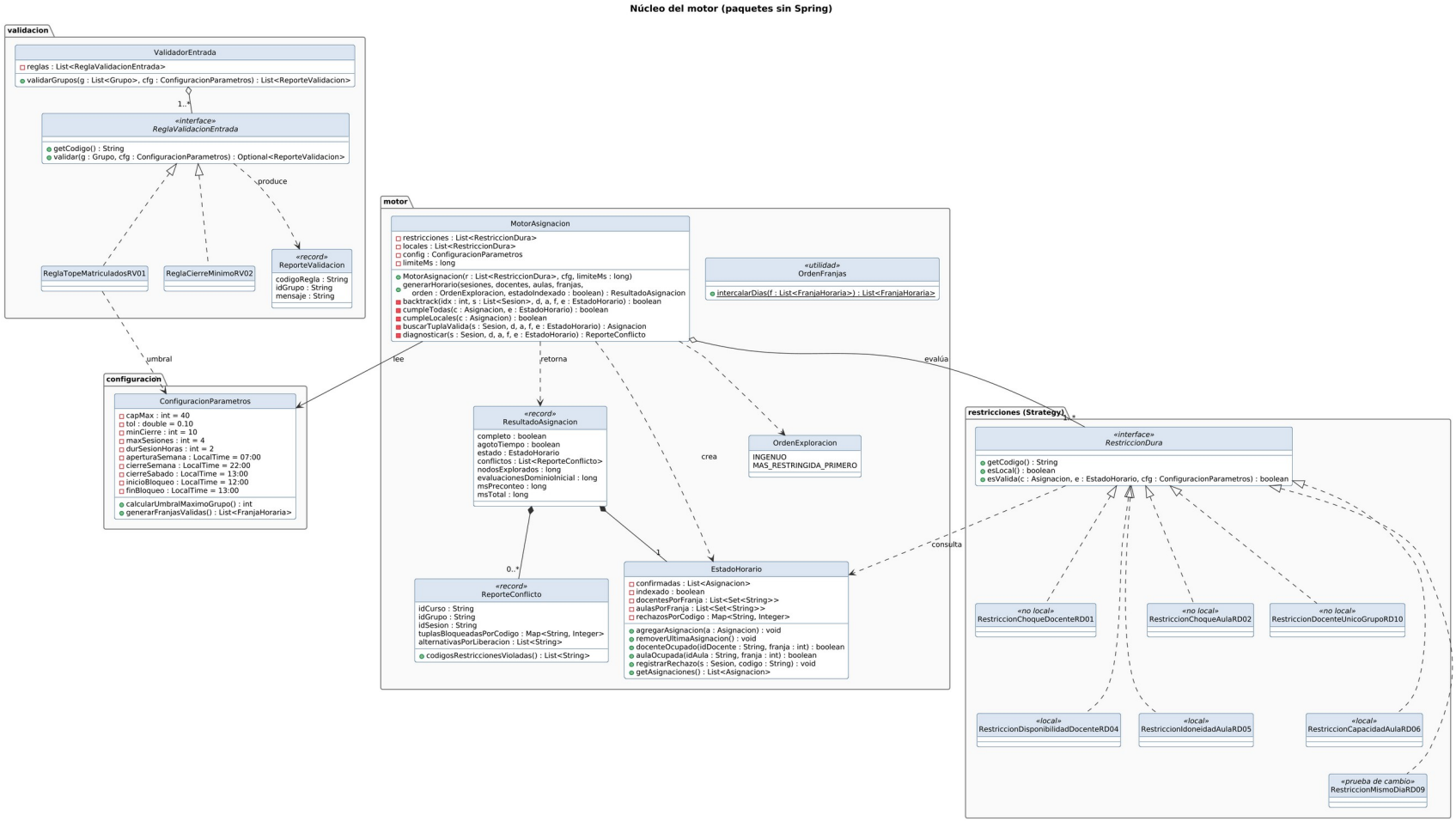
Figura	Artefacto	Fuente PlantUML
A1	Clases del núcleo del motor con tipos y patrón Strategy	clases_nucleo.puml
A2	Clases de las entidades del dominio con tipos	clases_entidades.puml
A3	Clases de las capas de aplicación, seguridad y persistencia	clases_aplicacion.puml
A4	Clases del módulo de usuarios por perfil	clases_usuarios.puml
B1	Actividades del algoritmo de asignación	actividades.puml
C1	Secuencia de una generación a través de la API	secuencia.puml
D1	Diagrama entidad relación final con 19 tablas	der.puml
E1	Componentes por capa	componentes.puml
E2	Despliegue en contenedores con Docker Compose	despliegue.puml

Anexo A. Diagramas de Clases

El diagrama de clases se presenta en cuatro figuras. La Figura A1 muestra el núcleo del motor, que no depende de Spring. La Figura A2 muestra las entidades del dominio que el núcleo manipula. La Figura A3 muestra cómo las capas de aplicación, seguridad y persistencia envuelven al núcleo para la versión web. La Figura A4 muestra el módulo de usuarios por perfil que se agregó al final del Módulo 3 con la gestión de catálogos, las cuentas y el portal del docente. Todas las clases del núcleo y del dominio son finales e inmutables salvo EstadoHorario, que cambia durante el backtracking, y ConfiguracionParametros, que se llena desde la base de datos.

Figura A1

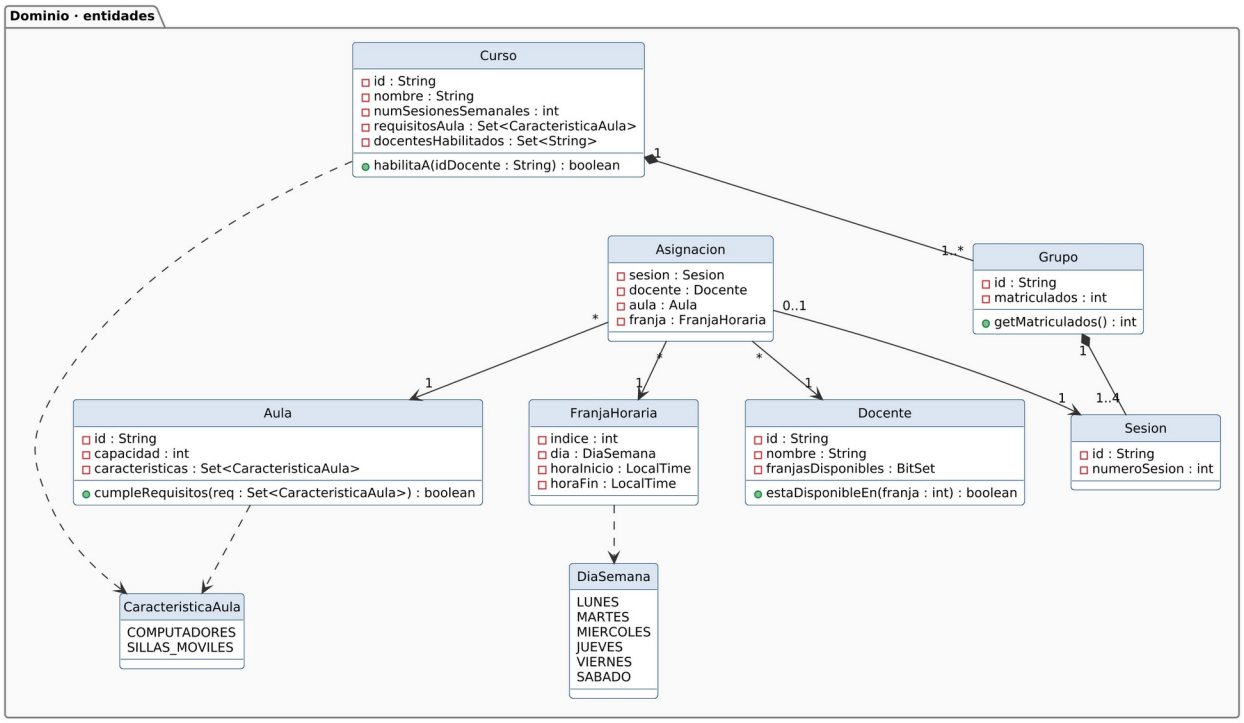
Clases del núcleo del motor y patrón Strategy



Nota. MotorAsignacion depende solo de la interfaz RestriccionDura. Los estereotipos local y no local indican qué devuelve esLocal. RestriccionMismoDiaRD09 existe solo para la prueba de modificabilidad y no está registrada en ConfiguracionMotor. Los registros (record) de Java se muestran con sus componentes.

Figura A2

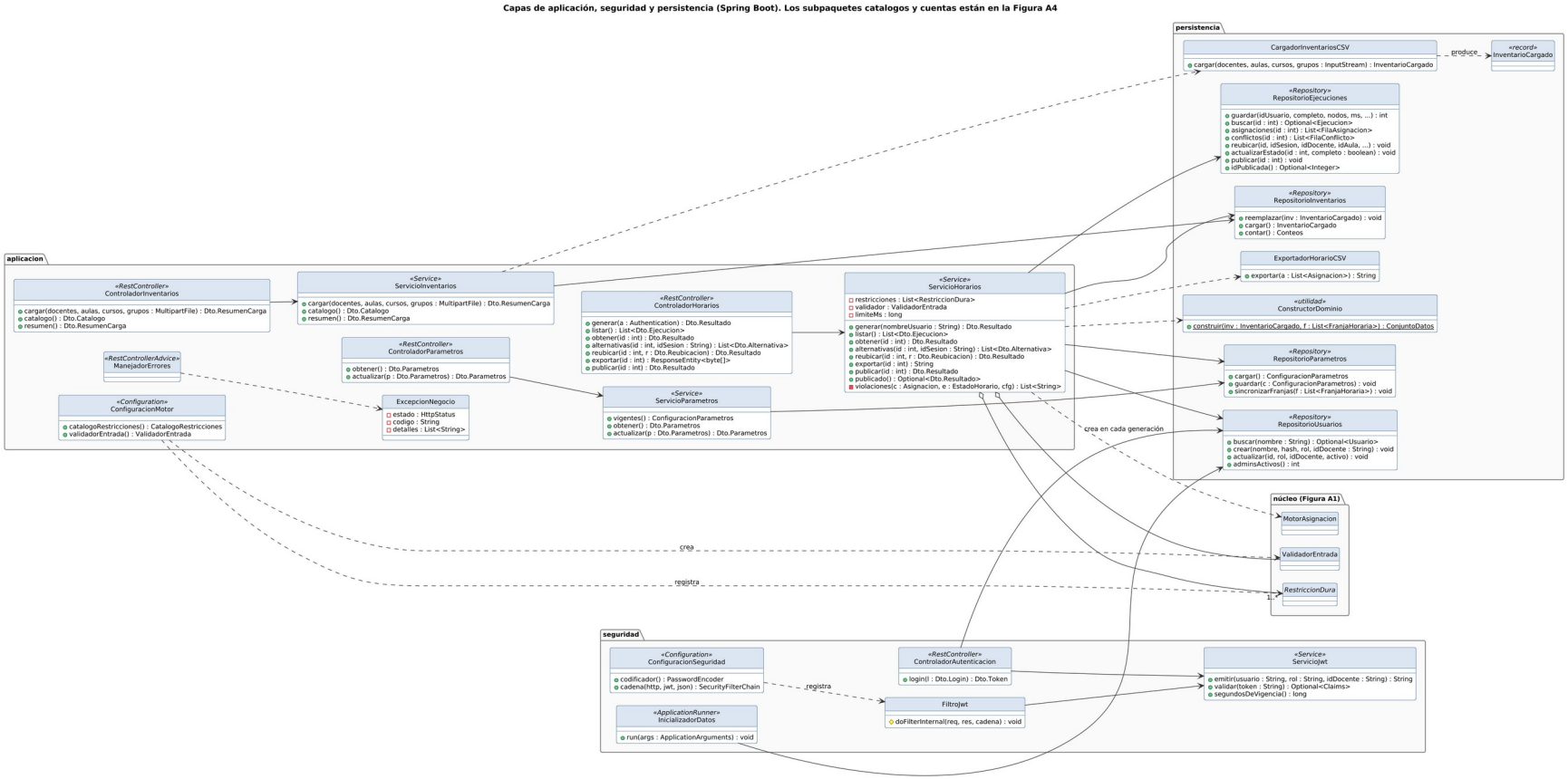
Clases de las entidades del dominio



Nota. Un curso compone uno o más grupos y cada grupo compone entre una y cuatro sesiones (RD-07). La disponibilidad del docente es un BitSet indexado por la franja vigente.

Figura A3

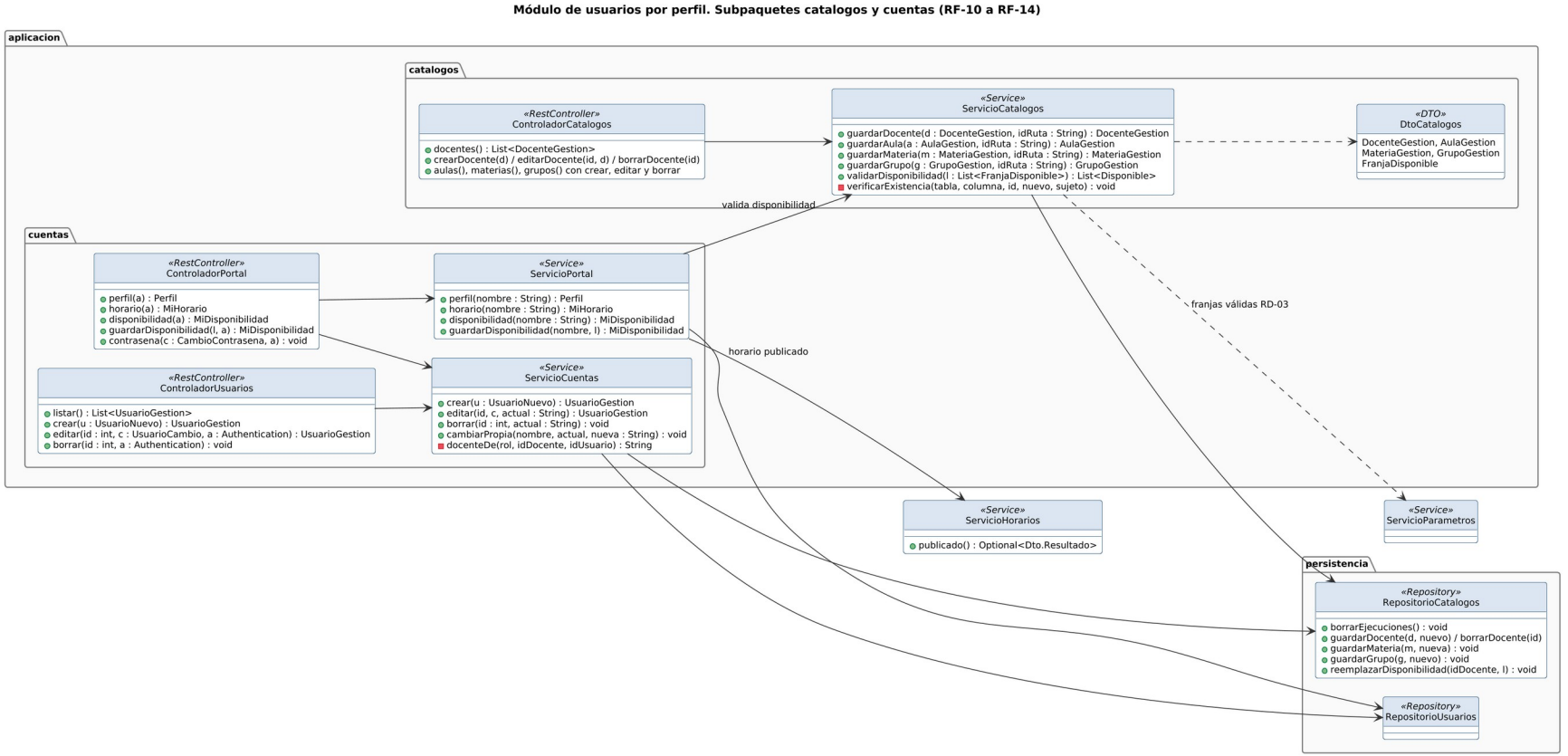
Clases de las capas de aplicación, seguridad y persistencia



Nota. Los controladores REST no tienen lógica y delegan en los servicios. ConfiguracionMotor es el único lugar donde se registran las reglas. Los repositorios usan JdbcTemplate con SQL explícito.

Figura A4

Clases del módulo de usuarios por perfil

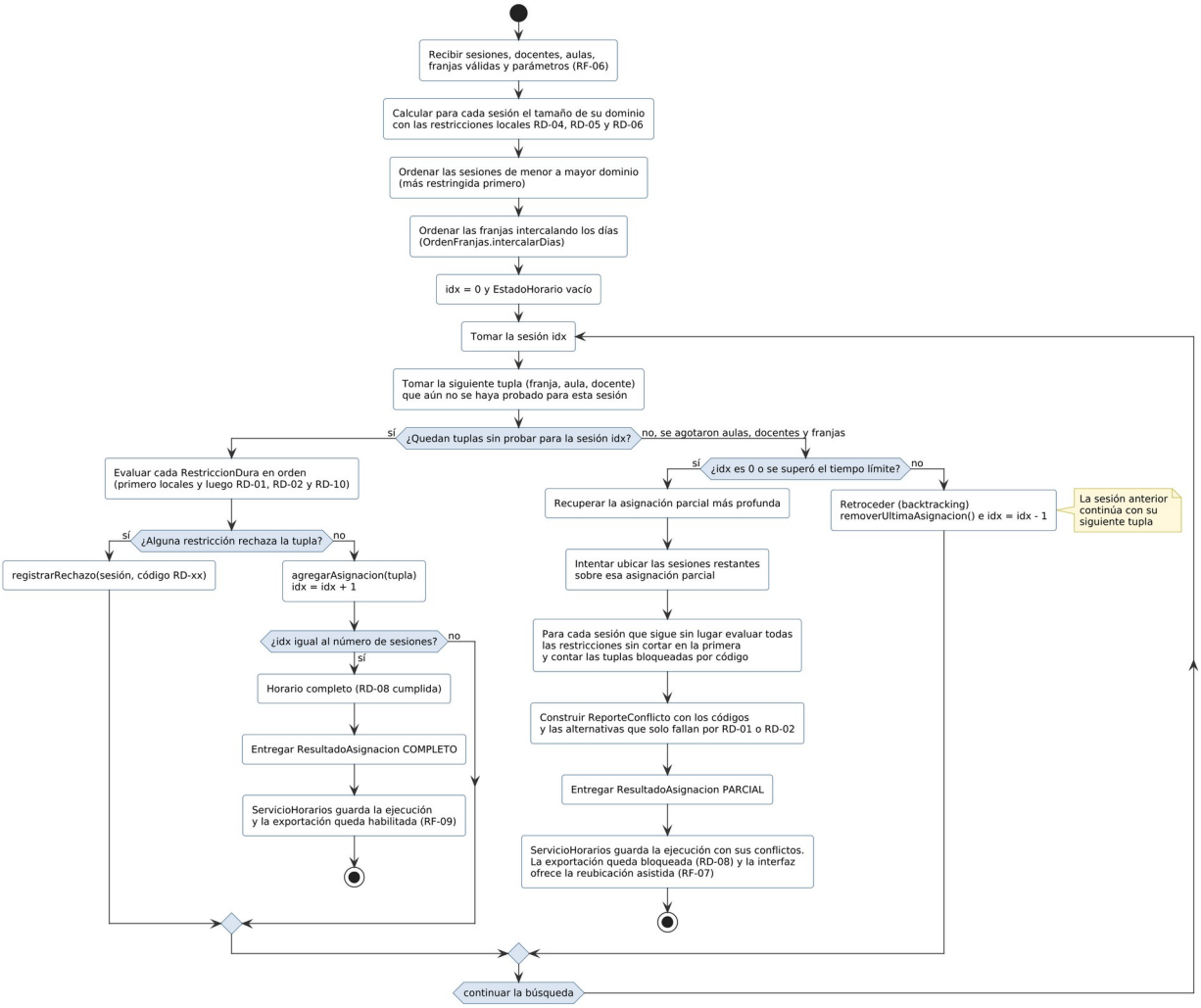


Nota. ServicioCatalogos valida con las mismas reglas del cargador CSV y cada cambio borra los horarios generados. ServicioPortal busca el docente de la cuenta autenticada en la base, así un docente solo ve sus propios datos. El núcleo del motor no cambió.

Anexo B. Diagrama de Actividades del Algoritmo

Figura B1

Backtracking con la sesión más restringida primero y diagnóstico de conflictos

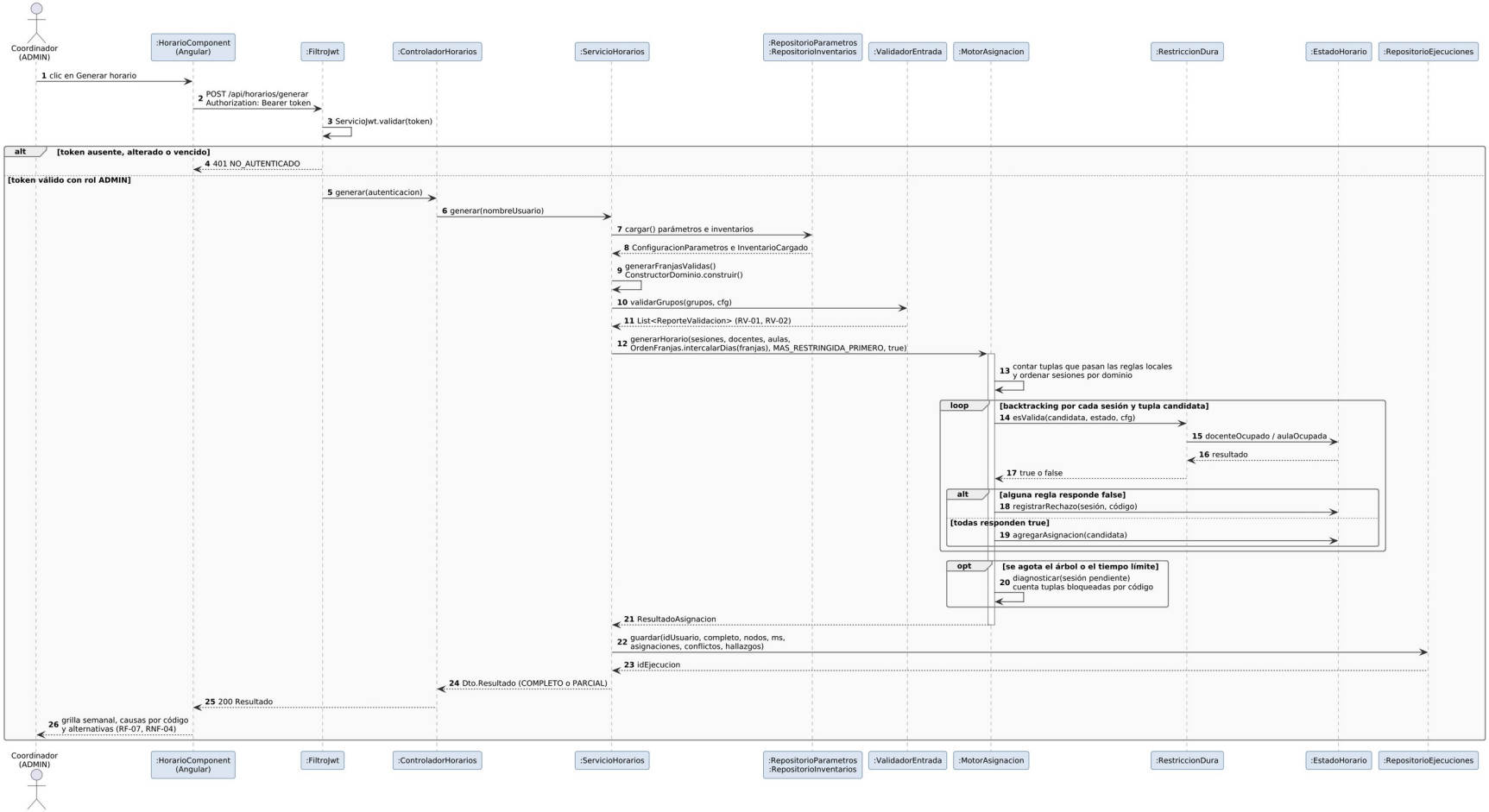


Nota. La rama de la derecha responde qué hace el sistema cuando se agotan las opciones de aula o de docente. El orden intercalado de franjas reparte las sesiones de un grupo en días distintos sin agregar una regla.

Anexo C. Diagrama de Secuencia

Figura C1

Generación de un horario a través de la API

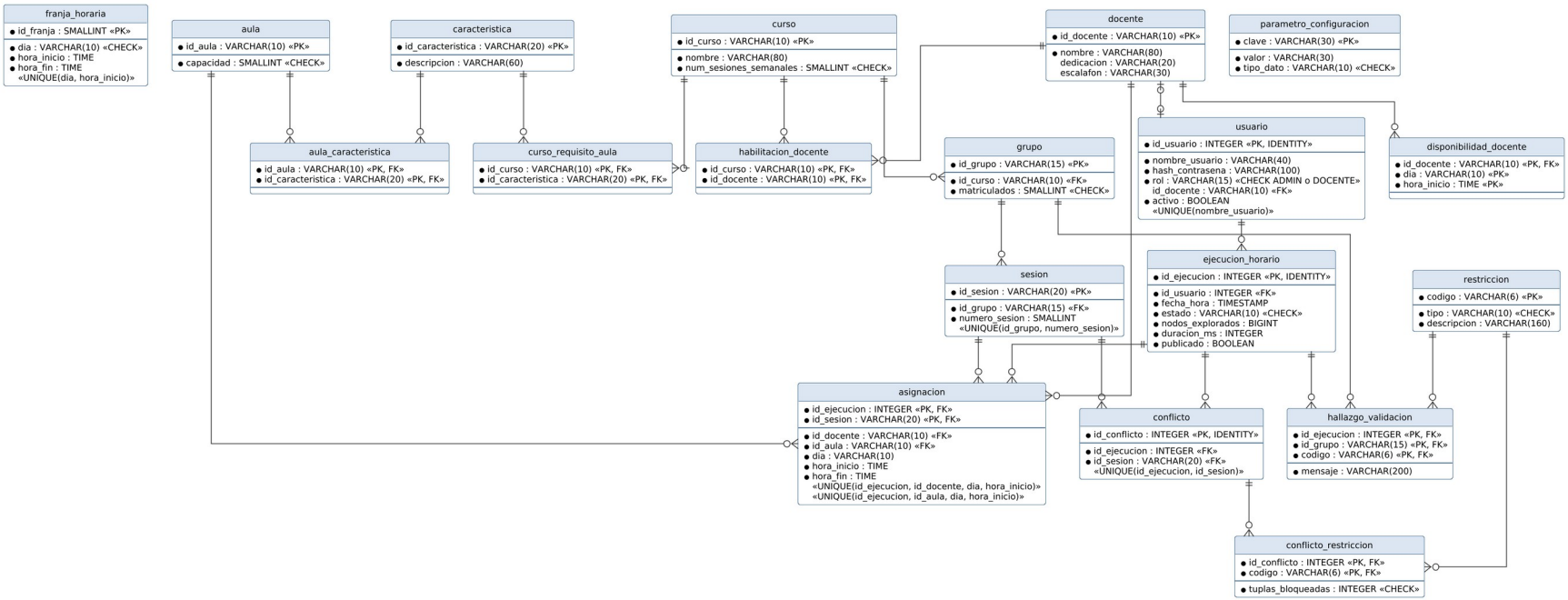


Nota. El filtro rechaza la petición con 401 si el token falta, está alterado o venció. ServicioHorarios crea un MotorAsignacion nuevo en cada generación con la lista de reglas que recibe de ConfiguracionMotor.

Anexo D. Modelo de Datos

Figura D1

Diagrama entidad relación final



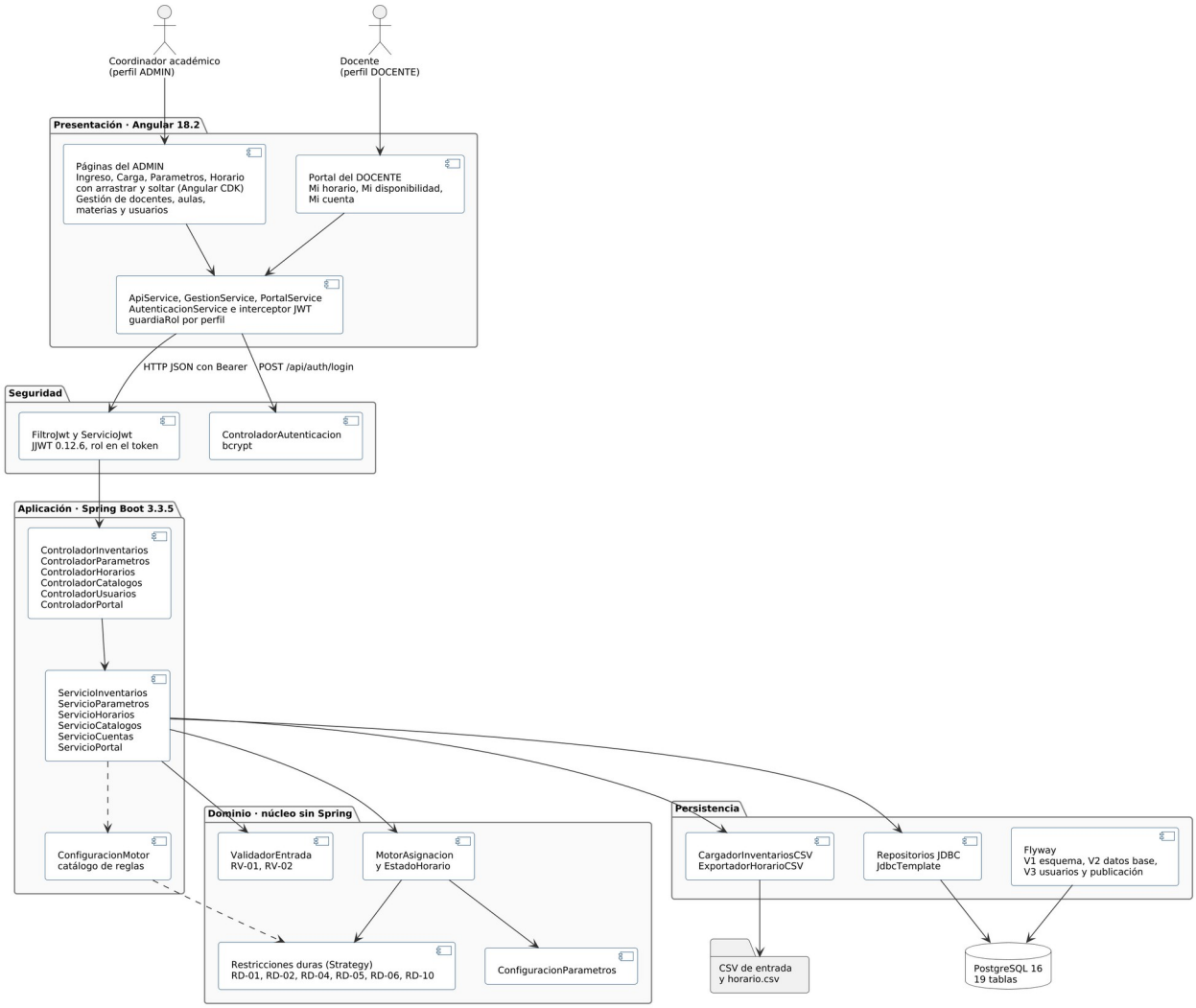
Nota. Generado desde backend/src/main/resources/db/migration/V1__esquema.sql con los cambios de V3__usuarios_y_publicacion.sql. Una cuenta DOCENTE queda ligada a un docente por id_docente y la columna publicado marca la única ejecución que ven los docentes. Notación de pata de gallo. El punto marca columnas obligatorias. Las dos restricciones UNIQUE de asignacion protegen RD-01 y RD-02 en la base de datos.

El modelo tiene 19 tablas en tercera forma normal. Frente al Módulo 2 cambió la forma de guardar el tiempo. La disponibilidad del docente y las asignaciones se guardan por día y hora de inicio y no por índice de franja. Así un cambio de la grilla en la pantalla de parámetros no deja datos que apunten a franjas que ya no existen. Por la misma razón las restricciones UNIQUE de asignacion quedaron sobre ejecución, docente, día y hora y sobre ejecución, aula, día y hora. El diccionario de datos completo de cada tabla está en el Documento 3. Manual Técnico (PN2_Grupo5_Doc3_ManualTecnico.pdf).

Anexo E. Diagramas de Componentes y Despliegue

Figura E1

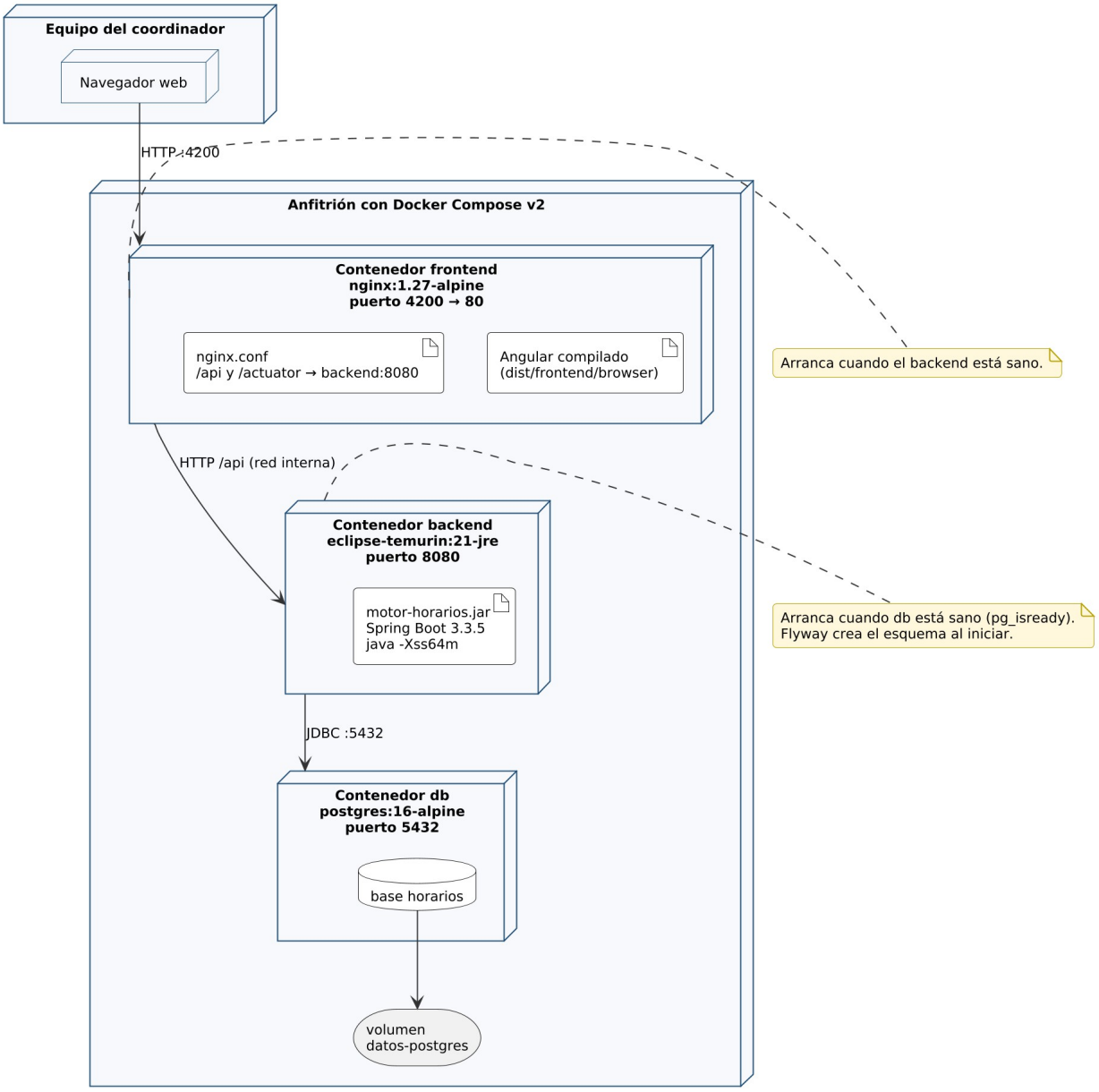
Componentes por capa



Nota. El núcleo corre en memoria dentro del backend. La persistencia lee los CSV una vez, guarda cada ejecución en PostgreSQL y Flyway crea el esquema al arrancar.

Figura E2

Despliegue en contenedores



Nota. Los tres contenedores se levantan con docker compose up --build. El orden de arranque lo controlan las verificaciones de salud de cada servicio.